Класификация на газове чрез данни от IoT сензорна мрежа

Този notebook е демонстрационен проект по дисциплината „Анализ на големи масиви данни и откриване на знания“,

реализиран от: **Димитър Николов, ф.н. 471222022**.

Темата на презентацията е **„Безжични сензорни мрежи за IoT“**, а проектът демонстрира как данни от сензорна система могат да бъдат използвани за обучение на ML модел.

Задачата е **таблична класификация**: по измервания от химически сензори моделът трябва да предскаже кой газ е бил засечен.

1. Инсталиране на библиотеки

`pip install -r requirements.txt`

2. Импортиране на нужните библиотеки

In [ ]:
```
from pathlib import Path
import zipfile
import urllib.request

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from fastai.tabular.all import *
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, ConfusionMatrixDisplay
from sklearn.ensemble import RandomForestClassifier
```

In [ ]:
```
# Папка, в която ще запазваме резултатите
RESULTS_DIR = Path("../results")
RESULTS_DIR.mkdir(exist_ok=True)

DATA_DIR = Path("../data/raw")
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

3. Зареждане на dataset-а

Използваме **Gas Sensor Array Drift Dataset** от UCI Machine Learning Repository.

Важно: пакетът `ucimlrepo` понякога връща грешка за този dataset, защото UCI не го предоставя като стандартизиран CSV import. Затова тук го сваляме директно като `.zip` файл от UCI и парсваме `.dat` файловете.

Форматът на редовете е подобен на LIBSVM:

```
1 1:15596.162100 2:1.868245 ... 128:-2.654529
```

Първото число е класът на газа, а следващите стойности са `feature:value`.

In [ ]:
```
ZIP_PATH = DATA_DIR / "gas_sensor_array_drift_dataset.zip"

DATASET_URLS = [
    "https://archive.ics.uci.edu/static/public/224/gas+sensor+array+drift+dataset.zip",
    "https://archive.ics.uci.edu/static/public/224/gas%2Bsensor%2Barray%2Bdrift%2Bdataset.zip",
]

GAS_LABELS = {
    1: "Ethanol",
    2: "Ethylene",
    3: "Ammonia",
    4: "Acetaldehyde",
    5: "Acetone",
    6: "Toluene",
}


def download_dataset_if_needed() -> None:
    """Сваля dataset-а само ако още не съществува локално."""
    if ZIP_PATH.exists():
        print(f"Dataset already exists: {ZIP_PATH}")
        return

    last_error = None

    for url in DATASET_URLS:
```

```python
        try:
            print(f"Downloading dataset from: {url}")
            urllib.request.urlretrieve(url, ZIP_PATH)
            print(f"Saved to: {ZIP_PATH}")
            return
        except Exception as error:
            last_error = error
            print("Download failed for this URL. Trying next one...")

    raise RuntimeError(
        "Dataset download failed. Open the UCI page manually, download the ZIP file, "
        f"and place it here: {ZIP_PATH}"
    ) from last_error


def parse_libsvm_line(line: str, batch_name: str) -> dict:
    """Парсва един ред от .dat файл във формат: class feature:value feature:value ..."""
    parts = line.strip().split()

    target_id = int(parts[0])
    row = {
        "target_id": target_id,
        "target": GAS_LABELS[target_id],
        "batch": batch_name,
    }

    for item in parts[1:]:
        feature_id, value = item.split(":")
        row[f"feature_{int(feature_id):03d}"] = float(value)

    return row


def load_gas_sensor_dataset() -> pd.DataFrame:
    """Сваля ZIP архива, чете batch*.dat файловете и връща pandas DataFrame."""
    download_dataset_if_needed()

    rows = []

    with zipfile.ZipFile(ZIP_PATH, "r") as archive:
        batch_files = [
            name for name in archive.namelist()
            if name.lower().endswith(".dat") and "batch" in Path(name).name.lower()
        ]

        batch_files = sorted(
            batch_files,
            key=lambda name: int("".join(ch for ch in Path(name).stem if ch.isdigit()))
        )

        print("Found batch files:")
        for file_name in batch_files:
            print(" -", file_name)

        for file_name in batch_files:
            batch_name = Path(file_name).stem

            with archive.open(file_name) as file:
                for raw_line in file:
                    line = raw_line.decode("utf-8").strip()

                    if line:
                        rows.append(parse_libsvm_line(line, batch_name))

    df = pd.DataFrame(rows)

    feature_columns = sorted([column for column in df.columns if column.startswith("feature_")])
    df = df[["target_id", "target", "batch"] + feature_columns]

    return df


df = load_gas_sensor_dataset()

print("Dataset shape:", df.shape)
df.head()
```

In [ ]:
```python
# Първите 5 реда от dataset-а
df.head()
```

In [ ]:
```python
# Класове газове в dataset-а
df[["target_id", "target"]].drop_duplicates().sort_values("target_id")
```

4. Подготовка на данните

Колоните са:

- `target_id` - числов ID на класа;
- `target` - името на газа, което моделът ще предсказва;
- `batch` - batch/времева група от dataset-а;
- `feature_001` до `feature_128` - числови характеристики от сензорите.

За обучението използваме само `feature_001` до `feature_128` като входни данни. Не използваме `batch` като вход, за да не помагаме изкуствено на модела чрез времевата група.

In [ ]:
```
feature_columns = [column for column in df.columns if column.startswith("feature_")]

print("Брой features:", len(feature_columns))
print("Dataset shape:", df.shape)

df[["target", "batch"] + feature_columns[:5]].head()
```

In [ ]:
```
# Проверка за липсващи стойности
missing_values = df.isna().sum().sum()
print("Общ брой липсващи стойности:", missing_values)
```

In [ ]:
```
# Проверяваме разпределението на класовете
class_counts = df["target"].value_counts().sort_index()
class_counts
```

In [ ]:
```
# Визуализация на разпределението на класовете
class_counts.plot(kind="bar")
plt.title("Разпределение на класовете")
plt.xlabel("Клас на газ")
plt.ylabel("Брой записи")
plt.tight_layout()
plt.savefig(RESULTS_DIR / "class_distribution.png")
plt.show()
```

5. Train/validation разделяне

Разделяме данните на:

- training set - използва се за обучение
- validation set - използва се за оценка по време на обучението

Използваме `stratify`, за да запазим подобно разпределение на класовете в двете части.

In [ ]:
```
train_df, valid_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df["target"]
)

print("Train shape:", train_df.shape)
print("Validation shape:", valid_df.shape)
```

6. Baseline модел: Random Forest

Първо обучаваме класически ML модел - **Random Forest Classifier**.

Той служи като baseline, с който можем да сравним FastAI модела.

In [ ]:
```
X_train = train_df[feature_columns]
y_train = train_df["target"]

X_valid = valid_df[feature_columns]
y_valid = valid_df["target"]

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_valid)

rf_accuracy = accuracy_score(y_valid, rf_predictions)
print(f"Random Forest Accuracy: {rf_accuracy:.4f}")
```

In [ ]:
```
rf_report = classification_report(y_valid, rf_predictions)
print(rf_report)
```

```
with open(RESULTS_DIR / "random_forest_results.txt", "w", encoding="utf-8") as file:
    file.write(f"Random Forest Accuracy: {rf_accuracy:.4f}\n\n")
    file.write(rf_report)
```

In [ ]:
```
ConfusionMatrixDisplay.from_predictions(y_valid, rf_predictions)
plt.title("Random Forest Confusion Matrix")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(RESULTS_DIR / "random_forest_confusion_matrix.png")
plt.show()
```

7. FastAI Tabular Learner

Сега обучаваме основния модел чрез **FastAI Tabular Learner**.

Тъй като всички входни характеристики са числови, ги подаваме като `cont_names`. Нямаме категорийни входни колони, затова `cat_names` е празен списък.

In [ ]:
```
# Обединяваме train и validation обратно, защото FastAI очаква един DataFrame + индекси за validation set.
combined_df = pd.concat([train_df, valid_df]).reset_index(drop=True)

valid_idx = list(range(len(train_df), len(combined_df)))

cont_names = [column for column in combined_df.columns if column != "target"]
cat_names = []
dep_var = "target"

print("Брой числови характеристики:", len(cont_names))
print("Target:", dep_var)
```

In [ ]:
```
dls = TabularDataLoaders.from_df(
    combined_df,
    y_names=dep_var,
    valid_idx=valid_idx,
    cat_names=cat_names,
    cont_names=cont_names,
    procs=[Normalize],
    y_block=CategoryBlock,
    bs=64
)

dls.show_batch(max_n=5)
```

In [ ]:
```
learn = tabular_learner(
    dls,
    layers=[200, 100],
    metrics=accuracy
)

learn
```

8. Обучение на FastAI модела

Обучаваме модела за 10 епохи.

Ако резултатът е слаб или loss-ът продължава да пада, може да се пробва с повече епохи, например 15 или 20.

In [ ]:
```
learn.fit_one_cycle(10, 1e-3)
```

In [ ]:
```
learn.recorder.plot_loss()
plt.title("FastAI Training Loss")
plt.tight_layout()
plt.savefig(RESULTS_DIR / "fastai_training_loss.png")
plt.show()
```

9. Оценка на FastAI модела

Изчисляваме accuracy върху validation set и визуализираме confusion matrix.

In [ ]:
```
fastai_results = learn.validate()
fastai_accuracy = fastai_results[1]

print(f"FastAI Validation Accuracy: {fastai_accuracy:.4f}")
```

In [ ]:
```
interp = ClassificationInterpretation.from_learner(learn)
interp.plot_confusion_matrix(figsize=(8, 8))
plt.title("FastAI Confusion Matrix")
plt.tight_layout()
```

```
plt.savefig(RESULTS_DIR / "fastai_confusion_matrix.png")
plt.show()
```

In [ ]:
```
# Запазваме обучен FastAI модел
learn.export(RESULTS_DIR / "fastai_gas_sensor_model.pkl")
print("Моделът е запазен в results/fastai_gas_sensor_model.pkl")
```

10. Сравнение на моделите

Тук сравняваме резултатите от Random Forest и FastAI модела.

In [ ]:
```
comparison = pd.DataFrame({
    "Model": ["Random Forest", "FastAI Tabular Learner"],
    "Accuracy": [rf_accuracy, float(fastai_accuracy)]
})

comparison
```

In [ ]:
```
comparison.plot(x="Model", y="Accuracy", kind="bar", legend=False)
plt.title("Сравнение на моделите")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(RESULTS_DIR / "model_comparison.png")
plt.show()
```

11. Извод

Проектът демонстрира как данни от IoT сензорна система могат да бъдат използвани за автоматично разпознаване на различни газове.

Този подход има реално приложение при:

- мониторинг на въздуха;
- индустриална безопасност;
- ранно откриване на опасни газове;
- анализ на данни от безжични сензорни мрежи;
- автоматизирано вземане на решения в IoT системи.